



Q Search

CNN with ImageDataGenerator.flow_from_dataframe

Notebook Input Output Logs Comments (3)

Found 84136 images belonging to 2 classes.
Train generator created
Found 28463 images belonging to 2 classes.
Validation generator created
Found 55897 images belonging to 2 classes.
Test generator created
Generators created

28 Copy & Edit 93

Table of Contents

- 1. Introduction
- 2. Quick EDA
- 3. Data preparation
- 4. Create and train the model
- 5. Evaluate trained model
- 5. Submission

4. Create and train the model

Put all creation and training code in one class. Will experiment with model architecture later.

```
In [7]: class ModelTrainer:
    """
    Create and fit the model
    """

    def __init__(self, generators):
        self.generators = generators
        self.img_width = generators.img_size[0]
        self.img_height = generators.img_size[1]

    def create_model_small(self):
        """
        Build CNN model using img_width, img_height from fields.
        """
        model=Sequential()
        model.add(Conv2D(16, kernel_size=3, input_shape=(self.img_width, self.img_height,3), activation='relu', padding='same'))
        model.add(MaxPooling2D(2))
        model.add(Conv2D(32, kernel_size=3, activation='relu', padding='same'))
        model.add(Dropout(0.1))
        model.add(Flatten())
        model.add(Dense(64, activation = 'relu'))
        # 1 y label
        model.add(Dense(2, activation='softmax'))
        model.compile(optimizer=Adam(lr=0.0001), loss='binary_crossentropy', metrics=['accuracy'])
        return model

    def create_model(self,
                    kernel_size = (3,3),
                    pool_size= (2,2),
                    first_filters = 32,
                    second_filters = 64,
                    third_filters = 128,
                    first_dense=256,
                    second_dense=128,
                    dropout_conv = 0.3,
                    dropout_dense = 0.3):

        model = Sequential()
        # First conv filters
        model.add(Conv2D(first_filters, kernel_size, activation = 'relu', padding="same",
                        input_shape = (self.img_width, self.img_height,3)))
        model.add(Conv2D(first_filters, kernel_size, padding="same", activation = 'relu'))
        model.add(Conv2D(first_filters, kernel_size, padding="same", activation = 'relu'))
        model.add(MaxPooling2D(pool_size = pool_size))
        model.add(Dropout(dropout_conv))

        # Second conv filter
        model.add(Conv2D(second_filters, kernel_size, padding="same", activation = 'relu'))
        model.add(Conv2D(second_filters, kernel_size, padding="same", activation = 'relu'))
        model.add(Conv2D(second_filters, kernel_size, padding="same", activation = 'relu'))
        model.add(MaxPooling2D(pool_size = pool_size))
        model.add(Dropout(dropout_conv))

        # Third conv filter
        model.add(Conv2D(third_filters, kernel_size, padding="same", activation = 'relu'))
        model.add(Conv2D(third_filters, kernel_size, padding="same", activation = 'relu'))
        model.add(Conv2D(third_filters, kernel_size, padding="same", activation = 'relu'))
        model.add(MaxPooling2D(pool_size = pool_size))
        model.add(Dropout(dropout_conv))

        model.add(Flatten())

        # First dense
        model.add(Dense(first_dense, activation = "relu"))
        model.add(Dropout(dropout_dense))
        # Second dense
        model.add(Dense(second_dense, activation = "relu"))
        model.add(Dropout(dropout_dense))

        # Out layer
        model.add(Dense(2, activation = "softmax"))

        model.compile(optimizer=Adam(lr=0.0001), loss='binary_crossentropy', metrics=['accuracy'])
        return model

    def train(self, model, toy):
        """
        Train the model
        """
        if toy:
            epochs=3
            steps_per_epoch=20
            validation_steps=2
        else:
            epochs=100
            steps_per_epoch=100
            #steps_per_epoch=30
            #steps_per_epoch=10
            validation_steps=5

        # We'll stop training if no improvement after some epochs
        earlystopper = EarlyStopping(monitor='val_acc', patience=10, verbose=1)
        reduce_lr = ReduceLROnPlateau(monitor='val_acc', factor=0.5, patience=2,
                                      verbose=1, mode='max', min_lr=0.00001)

        # Save the best model during the training
        checkpointer = ModelCheckpoint('best_model1.h5',
                                      monitor='val_acc',
                                      verbose=1
                                      ,save_best_only=True
                                      ,save_weights_only=True)

        # Train
        training = model.fit_generator(generator=self.generators.train_generator
                                      ,epochs=epochs
                                      ,steps_per_epoch=steps_per_epoch
                                      ,validation_data=self.generators.val_generator
                                      ,validation_steps=validation_steps
                                      ,callbacks=[earlystopper, checkpointer, reduce_lr])

        # Get the best saved weights
        model.load_weights('best_model1.h5')
        return training

# Create and train the model
trainer = ModelTrainer(generators)

# model = trainer.create_model(kernel_size = (3,3),
#                             pool_size= (2,2),
#                             first_filters = 64,
#                             second_filters = 128,
#                             third_filters = 256,
#                             first_dense=256,
#                             second_dense=128,
#                             dropout_conv = 0.4,
#                             dropout_dense = 0.3)

model = trainer.create_model(kernel_size = (3,3),
                            pool_size= (2,2),
                            first_filters = 128,
                            second_filters = 256,
                            third_filters = 512,
                            first_dense=256,
                            second_dense=128,
                            dropout_conv = 0.3,
                            dropout_dense = 0.2)

model.summary()
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 64, 64, 128)	3584
conv2d_1 (Conv2D)	(None, 64, 64, 128)	147584
conv2d_2 (Conv2D)	(None, 64, 64, 128)	147584
max_pooling2d (MaxPooling2D)	(None, 32, 32, 128)	0
dropout (Dropout)	(None, 32, 32, 128)	0
conv2d_3 (Conv2D)	(None, 32, 32, 256)	295168
conv2d_4 (Conv2D)	(None, 32, 32, 256)	590880
conv2d_5 (Conv2D)	(None, 32, 32, 256)	590880
max_pooling2d_1 (MaxPooling2D)	(None, 16, 16, 256)	0
dropout_1 (Dropout)	(None, 16, 16, 256)	0